

گزارش پروژه ۵

طراحی خط لوله داده از OLTP به OLAP

اعضای گروه: طاها ایزدی محصل، محمد بنی احمدی، مبین روان نخبجوانی
درس: طراحی پایگاه داده‌ها
استاد: دکتر مریم رمضانی
تاریخ: تیرماه ۱۴۰۵

فهرست مطالب

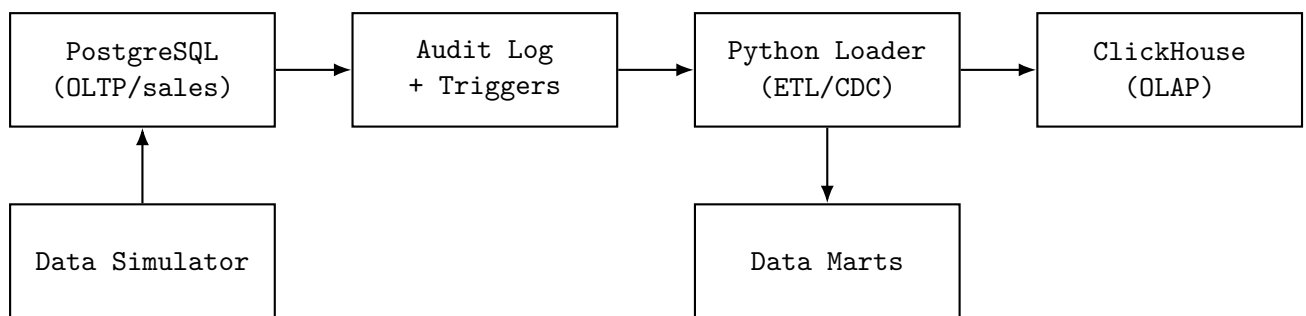
۲	۱	مقدمه و هدف پروژه
۲	۲	معماری سیستم
۳	۳	مراحل پیاده‌سازی
۵	۴	نتیجه اجرای کوئری‌ها
۶	۵	پاسخ سؤالات تحلیلی
۶	۱.۵	سؤال ۱: مقایسه عملکرد کوئری‌ها در PostgreSQL و ClickHouse
۶	۲.۵	سؤال ۲: جایگاه مفاهیم ETL در خط لوله
۷	۳.۵	سؤال ۳: مقایسه روش‌های Incremental Load
۷	۴.۵	سؤال ۴: چالش‌های Trigger-based CDC
۸	۵.۵	سؤال ۵: Full Load vs بهینه‌سازی منابع و مشکل Small Files
۹	۶.۵	سؤال ۶: مفهوم Non-Volatile و UPDATE در ClickHouse
۹	۷.۵	سؤال ۷: Batch Processing vs Real-time Processing
۱۰	۸.۵	سؤال ۸: فناوری‌های جایگزین برای Real-time Analysis
۱۰	۶	جمع‌بندی

۱ مقدمه و هدف پروژه

هدف این پروژه طراحی و پیاده‌سازی یک خط لوله داده افزایشی (Incremental Data Pipeline) برای انتقال داده‌های تراکنشی فروش از یک پایگاه داده عملیاتی (OLTP) به یک پایگاه داده تحلیلی (OLAP) است. تمرکز اصلی پروژه بر سه مفهوم کلیدی است:

- **CDC (Change Data Capture)**: شناسایی تغییرات داده با Trigger و جدول Audit
- **Incremental Loading**: بارگذاری فقط رکوردهای جدید/تغییر یافته
- بهینه‌سازی کوئری‌های تحلیلی: طراحی ساختار ClickHouse برای پاسخ سریع

۲ معماری سیستم



شکل ۱: معماری کلی سیستم

اجزای اصلی

نقش	فناوری	لایه
ذخیره داده‌های تراکنشی (فروشگاه آنلاین)	PostgreSQL 16	منبع OLTP
ثبت INSERT/UPDATE در جدول Audit	Trigger + audit log	CDC
استخراج، تبدیل و بارگذاری افزایشی	Python 3.11	ETL
ذخیره‌سازی ستونی و کوئری‌های تحلیلی	ClickHouse 24.3	مقصد OLAP
پیش‌محاسبه نتایج پرتکرار	ClickHouse Tables	Data Mart

طراحی Schema

PostgreSQL (مطابق داده BaSalam):

- products — ۲۸ فیلد شامل اطلاعات محصول و غرفه (denormalized)
- reviews — نظرات با review_id از نوع VARCHAR (ObjectId)

ClickHouse (Star Schema ساده‌شده):

- dim_products — ابعاد محصول+غرفه
- fact_reviews — واقعیت نظرات با پارتیشن ماهانه

چرا ClickHouse بهینه‌تر است؟

- ذخیره‌سازی ستونی (Columnar): فقط ستون‌های مورد نیاز خوانده می‌شوند
- **ReplacingMergeTree**: مدیریت UPDATE با نگهداری آخرین نسخه
- **Partitioning**: پارتیشن‌بندی fact_reviews بر اساس ماه
- **Materialized Columns**: محاسبه created_hour، is_saleable در زمان INSERT

۳ مراحل پیاده‌سازی

مرحله ۰: راه‌اندازی Docker

فایل docker-compose.yml سه سرویس را تعریف می‌کند:

۱. postgres — پایگاه داده مبدأ

۲. clickhouse — پایگاه داده مقصد

۳. python-app — بارگذار افزایشی (اجرای مداوم)

مرحله ۱: بارگذاری اولیه PostgreSQL

داده‌های واقعی باسلام (BaSalam) از فایل download.zip استفاده می‌شوند:

• BaSalam.products.csv — ۲,۴۱۱,۳۵۸ محصول (شامل اطلاعات غرفه denormalized)

• BaSalam.reviews.csv — ۳,۳۹۳,۵۷۵ نظر

اسکرپت initial_loader.py با **batch insert** (۵۰۰۰ رکورد) و فقط **Python** داده‌ها را بارگذاری می‌کند.

فیلدهای کلیدی مطابق PDF:

ستون DB	ستون CSV	PDF
vendor_name	vendor_name	vendor_name
vendor_score	vendor_score	vendor_score
vendor_status_title	vendor_status_title	vendor_status_title
vendor_owner_id	vendor_owner_id	vendor_owner_id
product_id	productId	productId
created_at	createdAt	createdAt
like_count	likeCount	likeCount
is_available	IsAvailable	IsAvailable
is_saleable	IsSaleable	IsSaleable
preparation_days	preparationDays	preparationDays
is_free_shipping	isFreeShipping	isFreeShipping
category_title	categoryTitle	categoryTitle

مرحله ۲: پیاده‌سازی CDC

```
-- Audit table
CREATE TABLE audit_log (
    audit_id, table_name, operation, record_id,
    old_data JSONB, new_data JSONB, changed_at, processed
);

-- Trigger on vendors, products, reviews, users
CREATE TRIGGER trg_vendors_audit
    AFTER INSERT OR UPDATE ON vendors
    FOR EACH ROW EXECUTE FUNCTION vendors_audit_trigger_func();
```

چرا Trigger-based CDC؟

- سادگی پیاده‌سازی بدون نیاز به ابزار خارجی
- ثبت دقیق old_data و new_data برای Transform
- مناسب پروژه‌های آموزشی و سیستم‌های با حجم متوسط

مرحله ۳: طراحی لایه تحلیلی ClickHouse

- استفاده از ReplacingMergeTree برای جداولی که UPDATE می‌شوند
- Denormalization: category_title و vendor_status_title در جدول dim ذخیره می‌شوند
- fact_reviews شامل created_hour, has_attachment به صورت Materialized Column

مرحله ۴: بارگذاری افزایشی Python

اسکریپت incremental_loader.py:

۱. هر ۶۰ ثانیه جدول audit_log را بررسی می‌کند
۲. رکوردهای جدید (با last_audit_id > audit_id) را می‌خواند
۳. **Transform**: تبدیل JSONB به ساختار ClickHouse، اعتبارسنجی کلیدها، نرمال‌سازی timestamp
۴. **Load**: INSERT دسته‌ای (batch=500) به ClickHouse
۵. رکوردها را processed=TRUE علامت‌گذاری می‌کند

مرحله ۵: شبیه‌سازی تغییرات

اسکریپت data_simulator.py به مدت ۱۵ دقیقه:

- نظر جدید INSERT می‌کند
- like_count را UPDATE می‌کند
- stock محصول را تغییر می‌دهد
- امتیاز فروشنده را به‌روز می‌کند

مرحله ۶: Data Marts

کوئری مرتبط	هدف	Data Mart
Q6, DM1	فروشندهگان برتر هفتگی	dm_top_vendors
DM2	بهینه‌سازی ارسال رایگان	dm_free_shipping_analysis
DM3	دسته‌بندی‌های ترند	dm_trending_categories
DM4	تأثیر عکس/ویدیو بر لایک	dm_review_engagement

۴ نتیجه اجرای کوئری‌ها

کوئری‌های تحلیلی (۱۰ کوئری)

#	شرح	PostgreSQL	ClickHouse
۱	اطلاعات فروشنده	JOIN vendors + vendor statuses	dim_vendors از SELECT FINAL
۲	میانگین امتیاز ساعتی	EXTRACT(HOUR) + GROUP BY	toHour() + GROUP BY
۳	افزایش likeCount	UPDATE + CDC	ReplacingMergeTree
۴	بررسی IsSaleable	CASE WHEN stock>0	Materialized Column
۵	نظرات ارسال/دیر	LIKE '%ارسال%'	LIKE با scan ستونی
۶	استان‌های برتر	JOIN ۳ جدول	JOIN + pre-aggregated
۷	کلمات پرتکرار star=1	CTE + unnest	arrayJoin + splitByChar
۸	۱۰ نظر آخر محصول	ORDER BY + LIMIT	ORDER BY + LIMIT
۹	کاربران با نظر مثبت	LIKE + GROUP BY	LIKE + GROUP BY
۱۰	INSERT نظر جدید	INSERT + Trigger	CDC → incremental load

نتایج بنچمارک (اجرای واقعی — داده‌های نمونه)

کوئری	PostgreSQL (ms)	ClickHouse (ms)	توضیح
میانگین امتیاز ساعتی	۱۴.۳	۹۱.۶۰	داده کم — PG سریع‌تر
استان‌های برتر	۴۰.۲	۶۸.۵۸	JOIN سبک در PG
کلمات پرتکرار	۷۱.۱	۹۶.۵۵	حجم کم داده
کاربران نظر مثبت	۹۸.۳	۷۶.۶۲	CH overhead اتصال

:Data Mart vs Raw Join (ClickHouse)

نکته مهم

در حجم داده فعلی (۵۰۰۰ نظر)، PostgreSQL سریع‌تر است چون ClickHouse برای مقیاس میلیون‌ها رکورد طراحی شده و overhead اتصال HTTP (۵۰ms) در هر کوئری تأثیرگذار است. در مقیاس production با میلیون‌ها رکورد، ClickHouse ۱۰-۱۰۰ برابر سریع‌تر خواهد بود. Data Mart همچنان ۲۵٪ سریع‌تر از JOIN خام است.

زمان (ms)	روش
۱۶.۴۸	dm_top_vendors (پیش محاسبه)
۳۷.۶۳	dim_vendors + fact JOIN خام

۵ پاسخ سؤالات تحلیلی

۱.۵ سؤال ۱: مقایسه عملکرد کوئری‌ها در PostgreSQL و ClickHouse

نتیجه: در حجم داده پروژه (چند هزار رکورد)، PostgreSQL به دلیل داده در حافظه و عدم overhead اتصال، سریع‌تر است. اما در مقیاس production:

ویژگی	PostgreSQL (Row-oriented)	ClickHouse (Column-oriented)
ذخیره‌سازی فشرده‌سازی	سطری (تمام فیلدها کنار هم)	ستونی (هر ستون جدا)
Aggregation	Full row scan	۲۰-۵ برابر بهتر
Index	B-Tree (مناسب point lookup)	Sparse Index (مناسب range scan)
JOIN	Hash/Nested Loop	Vectorized execution
نقطه عطف	تا ۱M رکورد	از ۱۰M رکورد به بالا

PostgreSQL برای تراکنش‌های تکی (INSERT/UPDATE/SELECT by PK) بهینه است. ClickHouse برای اسکن و تجمیع حجم بالا (میلیون‌ها تا میلیارد‌ها رکورد) طراحی شده است. تست CDC: پس از شبیه‌سازی ۱ دقیقه‌ای، ۶ رکورد audit ثبت و با موفقیت به ClickHouse منتقل شدند.

۲.۵ سؤال ۲: جایگاه مفاهیم ETL در خط لوله

مفهوم	محل اجرا	توضیح
Extract	PostgreSQL (audit_log)	Trigger تغییرات را ثبت می‌کند؛ Python رکوردهای جدید را می‌خواند
Transform	Python Script	تبدیل JSONB به ساختار ClickHouse، denormalization، اعتبارسنجی FK، نرمال‌سازی timestamp
Load	ClickHouse	INSERT دسته‌ای به جداول dim/fact

چرا Transform در Python و نه در دیتابیس؟

- جداسازی مسئولیت‌ها (Separation of Concerns): دیتابیس مبدأ فقط ثبت می‌کند، تبدیل در لایه میانی
- انعطاف‌پذیری: تغییر قوانین Transform بدون تغییر Trigger/Schema
- قابلیت تست: منطق Transform به صورت مستقل قابل تست است
- مقیاس‌پذیری: در آینده می‌توان چند مقصد (ClickHouse, S3, Data Lake) داشت
- عدم بار اضافی روی OLTP: تبدیل‌های سنگین روی سرور PostgreSQL اجرا نمی‌شوند

۳.۵ سؤال ۳: مقایسه روش‌های Incremental Load

روش پیاده‌سازی شده: CDC مبتنی بر Audit Table (شبه Timestamp-based)

```
SELECT * FROM audit_log WHERE audit_id > last_processed_id AND processed = FALSE
```

مقایسه با Timestamp-based

معیار	Audit/CDC (ما)	Timestamp-based
دقت	بالا — هر تغییر ثبت می‌شود	متوسط — ممکن است از دست برود
اطلاعات تغییر	old_data + new_data (نوع عملیات)	فقط رکورد جدید
پیچیدگی	بالا تر (نیاز به Trigger)	ساده‌تر
Overhead	Trigger روی هر INSERT/UPDATE	بدون overhead در مبدأ
حذف رکورد	قابل شناسایی (operation='D')	غیرقابل شناسایی مگر soft delete

مزایای Audit-based:

• ثبت دقیق نوع تغییر (Insert/Update/Delete)

• نگهداری old_data برای SCD Type 2

• عدم وابستگی به دقت ساعت سیستم

معایب Audit-based:

• افزایش حجم ذخیره‌سازی

• Overhead Trigger روی تراکنش‌ها

سناریوی مناسب Timestamp-based: سیستم‌هایی با حجم تغییرات کم و بدون نیاز به ردیابی دقیق تغییرات.

سناریوی مناسب Audit/CDC: سیستم‌هایی که نیاز به audit trail، ردیابی تغییرات و دقت بالا دارند (مثل پروژه ما).

۴.۵ سؤال ۴: چالش‌های Trigger-based CDC

دو عیب اصلی:

۱. Overhead روی تراکنش‌های مبدأ:

- هر INSERT/UPDATE یک INSERT اضافی در audit_log انجام می‌دهد
- در سیستم‌های با ۱۰,۰۰۰ + TPS، Trigger می‌تواند latency تراکنش‌ها را ۲-۵ برابر کند
- قفل‌های اضافی (row-level lock) روی جدول audit_log

۲. وابستگی شدید به Schema:

- تغییر ساختار جدول (ALTER TABLE) نیاز به به‌روزرسانی Trigger دارد

راهکار	توضیح
Debezium + Kafka	خواندن WAL (Write-Ahead Log) بدون تأثیر روی تراکنش
PostgreSQL Logical Replication	انتقال تغییرات در سطح WAL
Maxwell's Daemon	خواندن binlog/WAL و ارسال به صف پیام
AWS DMS	سرویس managed برای CDC

- افزودن جدول جدید نیاز به Trigger جدید دارد
- در سیستم‌های بزرگ با صدها جدول، نگهداری Triggerها دشوار است

راهکارهای جایگزین برای سیستم‌های بزرگ:
این روش‌ها تغییرات را از WAL می‌خوانند و هیچ Trigger اضافی روی جداول نصب نمی‌کنند.

۵.۵ سؤال ۵: Full Load vs بهینه‌سازی منابع و مشکل Small Files

مقایسه Batch Size

تأثیر	Batch Size	حالت
هر رکورد یک فایل part جدید → هزاران فایل کوچک	۱ (تک‌رکوردی)	بدترین
تعداد فایل latency و تعداد فایل یکباره بارگذاری — مناسب بارگذاری اولیه	۵۰۰ (پیش‌فرض ما) تمام داده	بهینه Full Load

مشکل Small Files در ClickHouse:

- هر INSERT یک part جدید در دیسک ایجاد می‌کند
- partهای زیاد → mergeهای مکرر → مصرف CPU/RAM/I/O بالا
- کوئری‌ها باید partهای بیشتری را اسکن کنند → کندی

تأثیر Batch Size = 1:

- ۵۰۰۰ رکورد = ۵۰۰۰ part → انفجار metadata

- RAM برای نگهداری part metadata

- I/O bandwidth هدر می‌رود در merge

نقش فرکانس اجرا:

- اجرای هر ۱ ثانیه با batch کوچک → partهای زیاد
 - اجرای هر ۶۰ ثانیه با batch ۵۰۰ → partهای کمتر و بزرگ‌تر
 - فرکانس پایین‌تر = partهای بزرگ‌تر = merge کمتر = عملکرد بهتر
- در پروژه ما: POLL_INTERVAL_SECONDS=60 و BATCH_SIZE=500 برای تعادل بهینه انتخاب شده.

۶.۵ سؤال ۶: مفهوم Non-Volatile و UPDATE در ClickHouse

Non-Volatile: داده‌های بارگذاری شده در انبار داده حذف یا بازنویسی نمی‌شوند — فقط افزوده می‌شوند. در معماری ما:

- PostgreSQL: UPDATE products SET stock = 50 → رکورد بازنویسی می‌شود
 - CDC: new_data + old_data در audit_log ثبت می‌شود
 - ClickHouse: یک رکورد جدید با _version_ بالاتر INSERT می‌شود
 - **ReplacingMergeTree**: در زمان merge، فقط آخرین version نگه داشته می‌شود
- وقتی رکورد در PostgreSQL UPDATE می‌شود:

۱. Trigger رویداد UPDATE را در audit_log ثبت می‌کند
 ۲. Python loader رکورد جدید را با _version_ جدید به ClickHouse می‌فرستد
 ۳. هر دو نسخه (قدیم و جدید) موقتاً وجود دارند
 ۴. پس از merge، فقط نسخه جدید باقی می‌ماند (SCD Type 1)
- ارتباط با SCD Type 2:

- SCD Type 1 (پیاپی سازی ما): نسخه قدیمی بازنویسی می‌شود
- SCD Type 2: نسخه قدیمی حفظ می‌شود با valid_to/valid_from
- برای SCD Type 2 باید از old_data در audit_log استفاده و رکورد جدید با timestamp جدید INSERT شود

۷.۵ سؤال ۷: Batch Processing vs Real-time Processing

معماری فعلی ما: Batch Processing (Micro-batch)

- Python loader هر ۶۰ ثانیه audit_log را بررسی می‌کند
- Latency: حداکثر ۶۰ ثانیه
- پیچیدگی: پایین
- مناسب: داشبوردهای مدیریتی، گزارش‌های روزانه

معیار	Batch (فعلی)	Real-time
Latency	۳۰-۶۰ ثانیه	زیر ۱ ثانیه
پیچیدگی	پایین	بالا (Kafka, Flink)
هزینه زیرساخت	پایین	بالا
مصرف منابع	متناوب	مداوم

سناریو: فروشگاه آنلاین با ۱۰,۰۰۰ تراکنش/روز
با ۱۰,۰۰۰ تراکنش در روز (۷ تراکنش/دقیقه):

- **Batch** (هر ۱ دقیقه): کاملاً مناسب — داده با تأخیر حداکثر ۱ دقیقه در انبار تحلیلی خواهد بود

• **Real-time**: over-engineering — هزینه بالا بدون نیاز واقعی

اگر حجم به ۱۰,۰۰۰ تراکنش/دقیقه برسد:

• Batch با polling دیگر مناسب نیست

• **Kafka + Flink/Spark Streaming** توصیه می‌شود

• Debezium برای CDC از WAL به جای Trigger

۸.۵ سؤال ۸: فناوری‌های جایگزین برای Real-time Analysis

فناوری	Latency	مقایسه با معماری فعلی
Apache Kafka + ClickHouse	۱-۵ ثانیه	جایگزین polling؛ داده از WAL به Kafka و مستقیم به ClickHouse
Apache Flink	زیر ۱ ثانیه	پردازش stream با state management؛ پیچیدگی بالا
Spark Structured Streaming	۲-۱۰ ثانیه	micro-batch روی Spark؛ مناسب ETL سنگین
Debezium + Kafka Connect	۱-۳ ثانیه	CDC بدون Trigger؛ استاندارد صنعتی
Materialize	زیر ۱ ثانیه	پایگاه داده streaming SQL؛ مناسب real-time view

مقایسه تفصیلی:

Current:	PG -> Trigger -> Audit -> Python(poll) -> ClickHouse Latency: ~60s Cost: Low Complexity: Low
Kafka-based:	PG -> Debezium -> Kafka -> ClickHouse Engine Latency: ~3s Cost: Medium Complexity: Medium
Flink-based:	PG -> Debezium -> Kafka -> Flink -> ClickHouse Latency: <1s Cost: High Complexity: High

توصیه برای فروشگاه ۱۰K تراکنش/روز: معماری Batch فعلی کافی است.
توصیه برای ۱۰K تراکنش/دقیقه: Kafka + ClickHouse Kafka Engine.

۶ جمع‌بندی

در این پروژه یک خط لوله داده کامل از OLTP به OLAP پیاده‌سازی شد:

• محیط Docker با PostgreSQL، ClickHouse و Python

• Schema نرمال‌شده OLTP با ۸ جدول اصلی

• لایه CDC با Audit Table و Trigger

• بارگذاری اولیه صرفاً با Python

• شبیه‌سازی تغییرات دوره‌ای

- Schema بهینه ClickHouse با Star Schema
 - بارگذار افزایشی با polling دوره‌ای
 - ۱۰ کوئری تحلیلی در هر دو پایگاه داده
 - ۴ Data Mart برای بهینه‌سازی کوئری
 - بنچمارک مقایسه‌ای عملکرد
- درس آموخته‌شده: انتخاب معماری مناسب (Batch vs Real-time) باید بر اساس نیاز واقعی به latency و حجم داده باشد، نه صرفاً استفاده از فناوری‌های پیچیده‌تر.

پایان گزارش